SIS4

Image Classification using Pretrained Models

Deep Learning

1 Objective

The objective of this laboratory work is to:

- Understand the concept of transfer learning and pretrained models
- Learn to use ResNet architecture for image classification
- Work with PyTorch and torchvision libraries
- Implement image preprocessing and inference pipelines
- Analyze model predictions and confidence scores

2 Theoretical Background

2.1 ResNet Architecture

ResNet (Residual Network) is a deep convolutional neural network architecture introduced by He et al. in 2015. It uses skip connections (residual connections) to solve the vanishing gradient problem in very deep networks.

2.2 Transfer Learning

Transfer learning involves using a model pretrained on a large dataset (like ImageNet with 1000 classes) and applying it to new classification tasks. This approach saves training time and computational resources.

3 Prerequisites

required libraries:

```
pip install torch torchvision pillow requests matplotlib
```

4 Example Code

4.1 Basic Image Classification

```python
import torch
from torchvision import models, transforms
from PIL import Image
import requests

# Load pretrained ResNet model
model = models.resnet50(pretrained=True)
model.eval()  # Set to evaluation mode

# Define image preprocessing
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

# Load and preprocess image
img = Image.open("your_image.jpg")
img_tensor = preprocess(img)
img_batch = img_tensor.unsqueeze(0)  # Add batch dimension

# Make prediction
with torch.no_grad():
    output = model(img_batch)

# Get predicted class
_, predicted_idx = torch.max(output, 1)
print(f"Predicted class index: {predicted_idx.item()}")
```

4.2 Classification with Top-K Probabilities

```python
import torch
from torchvision import models, transforms
from PIL import Image
import requests

def classify_image(image_path, top_k=5):
    """
    Classify image and return top-k predictions with
   probabilities
    """
    # Load model
    model = models.resnet50(pretrained=True).eval()

```

```
    # Preprocess
    preprocess = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225]),
    ])

    # Load labels
    url = "https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt"
    labels = requests.get(url).text.split("\n")

    # Process image
    img = preprocess(Image.open(image_path)).unsqueeze(0)

    # Predict
    with torch.no_grad():
        output = model(img)

    # Get probabilities and top-k
    probs = torch.nn.functional.softmax(output[0], dim=0)
    top_probs, top_idxs = torch.topk(probs, top_k)

    # Format results
    results = []
    for prob, idx in zip(top_probs, top_idxs):
        results.append({
            'class': labels[idx.item()],
            'probability': prob.item()
        })

    return results

# Usage
predictions = classify_image("your_image.jpg", top_k=5)

for i, pred in enumerate(predictions, 1):
    print(f"{i}. {pred['class']:30s} {pred['probability']*100:.2f}%")
```

5 Tasks

5.1 Task 1: Basic Implementation

a) Download 5 different images from the internet (different categories: animals, vehicles, objects, etc.)

b) Implement the basic classification

c) Run classification on all 5 images and save the results

d) Create a table showing the predicted class for each image

5.2 Task 2: Top-K Predictions

a) Modify the code to return top-5 predictions with probabilities

b) For each of your 5 test images, display:

- The original image
- Top-5 predicted classes with confidence scores

5.3 Task 3: Compare Different ResNet Models

a) Implement classification using three different ResNet variants:

- ResNet-18: `models.resnet18(pretrained=True)`
- ResNet-50: `models.resnet50(pretrained=True)`
- ResNet-101: `models.resnet101(pretrained=True)`

b) Measure and compare:

- Inference time for each model
- Model size (number of parameters)
- Top-1 accuracy on your test images

c) Create a comparison table and discuss the trade-offs

5.4 Task 4: Error Analysis

a) Find at least 2 images where the model makes incorrect predictions

b) Analyze why the model failed (ambiguous image, poor quality, unusual angle, etc.)

6 Extra Task

Implement batch processing:

a) Create a function that processes multiple images in a single batch

b) Compare the inference time of batch processing vs. individual processing

c) Process at least 10 images and measure the speedup

Hint: Stack multiple preprocessed images using `torch.stack()`